

Step 1 — Connect to One MCP Server and List Its Tools — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `mcp` SDK calls, in the right order), **Advanced** (what actually goes wrong with a real server, and how to see it clearly instead of hanging). Read Basic first even if you already know `asyncio` — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The exact client pieces, and the order they go in](#)
- [Hint 2 — The plan, and almost the whole script](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The exact client pieces, and the order they go in

Basic Version

An MCP server, when you connect to it over stdio, is just another program running on your machine — your client starts it as a subprocess and talks to it over its stdin/stdout, the same way any two programs can pipe data to each other. Before you can ask it anything, there's a required handshake: you connect, then you *initialize* the connection, and only after that has it succeeded can you actually ask "what tools do you have?"

Things to use: - Something that starts the server subprocess and gives you two raw streams to talk over it. - Something that wraps those two streams into the actual MCP protocol. - A call that does the required handshake. - A call that asks for the tool list.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The real `mcp` Python SDK splits this into exactly the two layers Basic described.

`stdio_client(server_params)` is an async context manager — it launches the server as a subprocess and gives you back a `(read, write)` pair of raw streams. `ClientSession(read, write)` is a second async context manager, wrapped around those streams, that speaks the actual MCP protocol on top of them (requests, responses, matching them up). You need both, nested, before anything else works.

The exact pieces: - `from mcp import ClientSession, StdioServerParameters` - `from mcp.client.stdio import stdio_client` - `StdioServerParameters(command="python", args=["example_server.py"])` — describes *how* to launch the server; nothing connects yet, this just builds the launch spec. - `async with stdio_client(server_params) as (read, write):` — actually launches the subprocess and opens the raw streams. - `async with ClientSession(read, write) as session:` — wraps those streams in the protocol. - `await session.initialize()` — the required handshake; nothing else on `session` is safe to call before this returns successfully. - `await session.list_tools()` — returns an object with a `.tools` attribute: a list of tool objects, each with `.name`, `.description`, and `.inputSchema`. - All of this needs an event loop — wrap your `async def main(): ...` in `asyncio.run(main())` at the bottom of the file.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what actually happens when the command you hand `StdioServerParameters` is wrong — a typo'd filename, a script that isn't executable, Python not on the `PATH` the subprocess sees. Some of those fail fast and loud (a `FileNotFoundError`-style exception right when `stdio_client` tries to launch the process). Others are quieter and more dangerous: the process *starts*, but never sends a valid MCP handshake back — maybe it's not an MCP server at all, maybe it crashed after starting but before responding. Without a timeout, `await session.initialize()` on a case like that can simply hang, with no exception at all, forever. Step 1 doesn't need the timeout fix yet (that's Step 4's job, deliberately), but you should see this failure mode now, on purpose, so you recognize it later instead of assuming a hang is your own code's bug.

Also worth knowing now: a server that connects and initializes successfully but genuinely has zero tools registered is not an error — `list_tools()` returns a `.tools` list, and an empty list is a completely valid, successful response. Don't write code that treats "no tools" the same as "connection failed" — they're different situations, and Step 4's fallback logic needs to be able to tell them apart later.

If you don't have a real server yet, the minimal one from Doc06's Core Concepts is worth typing out yourself first — a `FastMCP` instance, one or two `@mcp.tool()`-decorated functions, and `mcp.run(transport="stdio")` at the bottom. That's a complete, real MCP server in well under 20 lines, and having one you wrote yourself makes it much easier to tell, when something goes wrong, whether the bug is in your client or your server.

Things to try before Hint 2: - Run your client against a working server, and separately against a deliberately broken command — compare what each one actually does (exception vs. hang) with your own eyes before reading further.

Difference between Basic, Intermediate, and Advanced: Basic names the two things you need (a raw connection, and a protocol layer on top) without the real API. Intermediate gives the exact `mcp` SDK calls, in the right nested order, for the case where everything works. Advanced is about the case where it *doesn't* — a bad command, a server that starts but never responds, and the completely valid case of a server with no tools at all — which is the difference between a script that only works against your one tested server and one that tells you clearly what actually happened when it doesn't.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole script

Basic Version

```
build the launch spec: command + args for the server

open the connection (this launches the subprocess):
    open a session on top of it:
        do the handshake (initialize)
        ask for the tool list
        for each tool: print its name, description, and input shape

run all of this inside an async main(), started with asyncio.run(main())
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole thing — the two nested `async with` blocks are the part worth typing yourself rather than just reading:

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def list_server_tools(command: str, args: list[str]) -> None:
    server_params = StdioServerParameters(command=command, args=args)
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            for tool in tools_result.tools:
                print(f"- {tool.name}: {tool.description}")
                # your turn: also print tool.inputSchema, formatted however
                # is easiest for you to read

if __name__ == "__main__":
    asyncio.run(list_server_tools("python", ["example_server.py"]))
```

Expected output, against a server with a `get_weather` tool: something like

```
- get_weather: Get the current weather for a specific city.
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the `inputSchema` printing yourself, and add the wrong-command test case from Hint 1's Advanced section:

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def list_server_tools(command: str, args: list[str]) -> None:
    server_params = StdioServerParameters(command=command, args=args)
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            if not tools_result.tools:
                print("Connected fine -- this server just has no tools registered")
                return
            for tool in tools_result.tools:
                print(f"- {tool.name}: {tool.description}")
                print(f"  input schema: {tool.inputSchema}")

async def main() -> None:
    print("== working server ==")
    await list_server_tools("python", ["example_server.py"])

    print("\n== deliberately wrong command ==")
    try:
        await list_server_tools("python", ["this_file_does_not_exist.py"])
    except Exception as e:
        # your turn: what specific exception type shows up here on your
        # machine? name it, don't just catch bare Exception in real code
        print(f"Failed as expected: {type(e).__name__}: {e}")

if __name__ == "__main__":
    asyncio.run(main())
```

Run this against your own server and confirm you see exactly what Hint 1's Advanced section described, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both only handle the happy path — a server that works, printed cleanly. Advanced adds the two things a real integration actually needs: handling the valid-but-empty tool list without treating it as an error, and deliberately triggering (and naming) the failure case, instead of only ever running this against a server you already know works.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)